

Amazon DynamoDB AWS Managed NoSQL

Chapter 06 - Sub-Chapter 04

NoSQL Databases Series

What is DynamoDB?	Serverless, single-digit ms latency at any scale
Data Model	Tables, Items, Partition Key, Sort Key
CRUD Operations	PutItem, GetItem, Query, UpdateItem
Secondary Indexes	GSI and LSI for flexible query patterns
Spring Integration	Enhanced Client, annotations, repository pattern
When to Use DynamoDB	Strengths, limits, real-world companies

1

What is Amazon DynamoDB?

Serverless key-value and document database at any scale

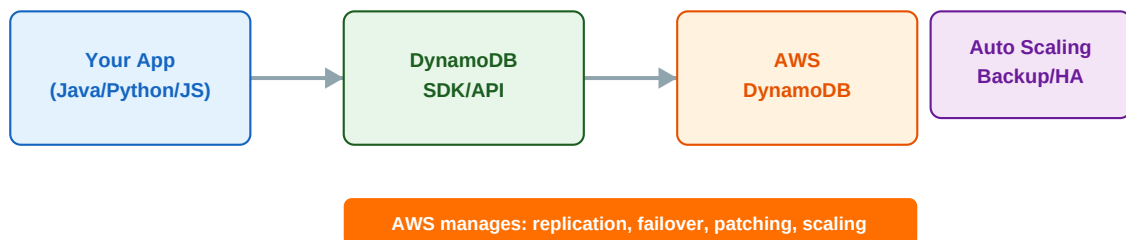
Picture this: it is Black Friday. Your e-commerce platform suddenly gets 10 million concurrent users. Your relational database buckles. Your ops team scrambles to provision more servers. Meanwhile, Amazon.com — which uses DynamoDB — handles the load automatically, invisibly, without a single manual intervention.

Amazon DynamoDB is a fully managed, serverless, key-value and document database. 'Fully managed' means AWS handles provisioning, patching, replication, failover, backups — everything. You write code, not infrastructure.

Key characteristics:

- **Single-digit millisecond latency** at any scale — reads and writes consistently fast whether you have 1 user or 100 million.
- **Serverless and auto-scaling:** No cluster to manage. Capacity scales up and down with traffic automatically (On-Demand mode).
- **Multi-region active-active:** Global Tables replicate data to multiple AWS regions in under a second.
- **Event-driven integration:** DynamoDB Streams emit change events — perfect for triggering Lambda functions.
- **ACID transactions:** Full ACID support across multiple items and tables (added 2018) — rare for a NoSQL database.
- **Pay-per-use pricing:** You pay for read/write capacity units consumed, not for idle servers.

DynamoDB Architecture — Fully Managed, Serverless



Real-world analogy: DynamoDB is like a magical filing cabinet. You tell it 'store this document under key X' and 'give me the document for key X' — and it does so in milliseconds, for any number of documents, without you ever worrying about the cabinet running out of drawers.

2 DynamoDB Data Model

Tables, Items, Attributes, and Primary Keys

DynamoDB's data model is deceptively simple at first glance but requires a different way of thinking compared to SQL. Let us build up from the basics.

Core concepts:

- **Table** — a collection of items. Similar to an SQL table but schema-free.
- **Item** — a single record. Like an SQL row, but each item can have different attributes.
- **Attribute** — a name-value pair. Like an SQL column, but not required on every item.
- **Partition Key (PK)** — mandatory. Determines which partition/node stores the item. Must be unique if used alone.
- **Sort Key (SK)** — optional. Combined with PK, enables range queries and 1:many relationships within a partition.

DynamoDB Data Model — Table, Items, Attributes

PK: user_id (Partition Key)	SK: order_id (Sort Key)	status	total	tags (Set)
alice	2024-01-15#uuid-1	CONFIRMED	120.50	{"prime","gift"}
alice	2024-01-16#uuid-2	SHIPPED	89.00	{"prime"}
bob	2024-01-15#uuid-3	PENDING	45.00	{}

Each item can have different attributes (schema-flexible)

Creating a table (AWS SDK for Java):

```
import software.amazon.awssdk.services.dynamodb.DynamoDbClient;
import software.amazon.awssdk.services.dynamodb.model.*;

DynamoDbClient dynamo = DynamoDbClient.builder()
    .region(Region.US_EAST_1)
    .build();

// Create orders table with composite primary key
CreateTableRequest request = CreateTableRequest.builder()
    .tableName("Orders")
    .billingMode(BillingMode.PAY_PER_REQUEST) // On-Demand – no capacity planning
    .attributeDefinitions(
        AttributeDefinition.builder()
            .attributeName("user_id").attributeType(ScalarAttributeType.S).build(),
        AttributeDefinition.builder()
            .attributeName("order_id").attributeType(ScalarAttributeType.S).build()
    )
    .keySchema(
        KeySchemaElement.builder()
            .attributeName("user_id").keyType(KeyType.HASH).build(), // Partition Key
        KeySchemaElement.builder()
            .attributeName("order_id").keyType(KeyType.RANGE).build() // Sort Key
    )
    .build();
```

```
.build();  
  
dynamo.createTable(request);
```

Schema flexibility: Notice we only define attributes that are part of the key. Other attributes (status, total, tags) are added freely per item — no ALTER TABLE needed.

3 CRUD Operations

PutItem, GetItem, Query, Scan, UpdateItem

PutItem — write a single item:

```
import software.amazon.awssdk.services.dynamodb.model.PutItemRequest;
import java.util.Map;

Map<String, AttributeValue> item = Map.of(
    "user_id", AttributeValue.fromS("alice"),
    "order_id", AttributeValue.fromS("2024-01-15#uuid-001"),
    "status", AttributeValue.fromS("CONFIRMED"),
    "total", AttributeValue.fromN("120.50"),
    "tags", AttributeValue.fromSs(List.of("prime", "gift")) // String Set
);

dynamo.putItem(PutItemRequest.builder()
    .tableName("Orders")
    .item(item)
    .conditionExpression("attribute_not_exists(order_id)") // prevent overwrites
    .build());
```

GetItem — retrieve one item by exact key:

```
import software.amazon.awssdk.services.dynamodb.model.GetItemRequest;
import software.amazon.awssdk.services.dynamodb.model.GetItemResponse;

GetItemResponse response = dynamo.getItem(GetItemRequest.builder()
    .tableName("Orders")
    .key(Map.of(
        "user_id", AttributeValue.fromS("alice"),
        "order_id", AttributeValue.fromS("2024-01-15#uuid-001")
    ))
    .build());

if (response.hasItem()) {
    String status = response.item().get("status").s();
    String total = response.item().get("total").n();
    System.out.printf("Order status: %s, total: $%s%n", status, total);
}
```

Query — all orders for a user, sorted (fast — uses partition):

```
import software.amazon.awssdk.services.dynamodb.model.QueryRequest;
import software.amazon.awssdk.services.dynamodb.model.QueryResponse;

// Get all orders for alice, sorted by order_id, newest first
QueryResponse result = dynamo.query(QueryRequest.builder()
    .tableName("Orders")
    .keyConditionExpression("user_id = :uid")
    .filterExpression("status = :st") // filter AFTER query (in-memory)
    .expressionAttributeValues(Map.of(
        ":uid", AttributeValue.fromS("alice"),
        ":st", AttributeValue.fromS("CONFIRMED")
    ))
    .scanIndexForward(false) // DESC sort on sort key
    .limit(10)
    .build());
```

```
result.items().forEach(item ->
    System.out.println(item.get("order_id").s() + " -> " + item.get("total").n())
);
```

UpdateItem — atomic field update:

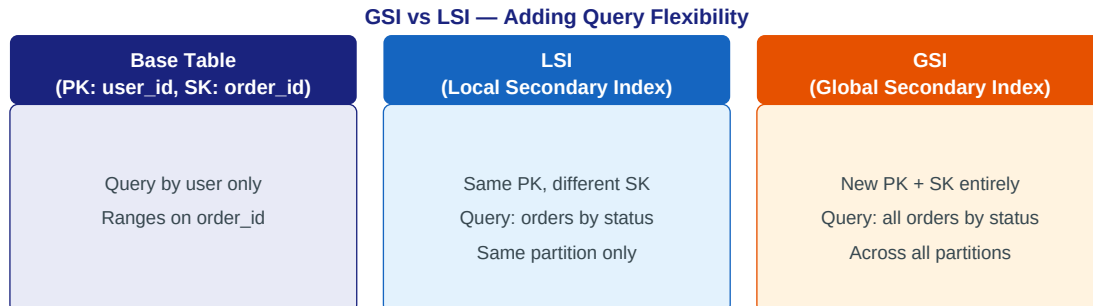
```
import software.amazon.awssdk.services.dynamodb.model.UpdateItemRequest;

dynamo.updateItem(UpdateItemRequest.builder()
    .tableName("Orders")
    .key(Map.of(
        "user_id", AttributeValue.fromS("alice"),
        "order_id", AttributeValue.fromS("2024-01-15#uuid-001")
    ))
    .updateExpression("SET #st = :newStatus, updated_at = :now")
    .conditionExpression("#st = :oldStatus") // optimistic locking
    .expressionAttributeNames(Map.of("#st", "status"))
    .expressionAttributeValues(Map.of(
        ":newStatus", AttributeValue.fromS("SHIPPED"),
        ":oldStatus", AttributeValue.fromS("CONFIRMED"),
        ":now",      AttributeValue.fromS(Instant.now().toString())
    ))
    .build());
// Throws ConditionalCheckFailedException if status was already changed
```

4 Secondary Indexes (GSI and LSI)

Query on non-key attributes

DynamoDB's biggest limitation: you can only query efficiently by primary key. Secondary indexes solve this — they are automatic projections of your table with a different key structure.



GSI example — query all orders by status across all users:

```
import software.amazon.awssdk.services.dynamodb.model.GlobalSecondaryIndex;
import software.amazon.awssdk.services.dynamodb.model.Projection;
import software.amazon.awssdk.services.dynamodb.model.ProjectionType;

// GSI defined at table creation (or added later)
GlobalSecondaryIndex statusIndex = GlobalSecondaryIndex.builder()
    .indexName("status-created-index")
    .keySchema(
        KeySchemaElement.builder()
            .attributeName("status").keyType(KeyType.HASH).build(), // new PK
        KeySchemaElement.builder()
            .attributeName("created_at").keyType(KeyType.RANGE).build() // new SK
    )
    .projection(Projection.builder()
        .projectionType(ProjectionType.ALL) // include all attributes
        .build())
    .build();

// Query the GSI -- find all CONFIRMED orders sorted by creation date
QueryResponse gsiResult = dynamo.query(QueryRequest.builder()
    .tableName("Orders")
    .indexName("status-created-index") // use the GSI
    .keyConditionExpression("status = :st AND created_at > :since")
    .expressionAttributeValues(Map.of(
        ":st", AttributeValue.fromS("CONFIRMED"),
        ":since", AttributeValue.fromS("2024-01-01")
    ))
    .build());
```

Cost of GSIs: Each GSI is essentially a copy of (part of) your table — it consumes additional write capacity every time the base table changes. Design GSIs carefully; aim for 5 or fewer per table.

5 Spring + DynamoDB Integration

Spring Data DynamoDB and Enhanced Client

There are two main approaches to use DynamoDB from Spring Boot: the AWS SDK Enhanced Client (annotation-based, type-safe) and Spring Data DynamoDB (community library). We will use the Enhanced Client — it is the AWS-recommended approach.

Maven dependency:

```
<!-- pom.xml -->
<dependency>
  <groupId>software.amazon.awssdk</groupId>
  <artifactId>dynamodb-enhanced</artifactId>
  <version>2.25.0</version>
</dependency>
<dependency>
  <groupId>io.awspring.cloud</groupId>
  <artifactId>spring-cloud-aws-dynamodb</artifactId>
  <version>3.1.0</version>
</dependency>
```

Entity class with Enhanced Client annotations:

```
import software.amazon.awssdk.enhanced.dynamodb.mapper.annotations.*;
import java.math.BigDecimal;
import java.time.Instant;
import java.util.Set;

@DynamoDbBean // marks this as a DynamoDB entity
public class Order {

    private String userId;
    private String orderId;
    private String status;
    private BigDecimal total;
    private Instant createdAt;
    private Set<String> tags;

    @DynamoDbPartitionKey // maps to Partition Key
    @DynamoDbAttribute("user_id")
    public String getUserId() { return userId; }

    @DynamoDbSortKey // maps to Sort Key
    @DynamoDbAttribute("order_id")
    public String getOrderId() { return orderId; }

    @DynamoDbSecondaryPartitionKey(indexNames = "status-created-index")
    public String getStatus() { return status; }

    // other getters/setters...
}
```

Configuration and Repository Service:

```
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import software.amazon.awssdk.enhanced.dynamodb.DynamoDbEnhancedClient;
import software.amazon.awssdk.enhanced.dynamodb.DynamoDbTable;
import software.amazon.awssdk.enhanced.dynamodb.TableSchema;
```



```

import software.amazon.awssdk.services.dynamodb.DynamoDbClient;
import org.springframework.stereotype.Repository;
import java.util.List;
import java.util.stream.Collectors;

@Configuration
public class DynamoConfig {
    @Bean
    public DynamoDbEnhancedClient enhancedClient(DynamoDbClient dynamo) {
        return DynamoDbEnhancedClient.builder().dynamoDbClient(dynamo).build();
    }
    @Bean
    public DynamoDbTable<Order> orderTable(DynamoDbEnhancedClient enhanced) {
        return enhanced.table("Orders", TableSchema.fromBean(Order.class));
    }
}

@Repository
public class OrderRepository {
    private final DynamoDbTable<Order> table;

    public OrderRepository(DynamoDbTable<Order> table) { this.table = table; }

    public void save(Order order) {
        table.putItem(order);
    }

    public Order findById(String userId, String orderId) {
        Key key = Key.builder()
            .partitionValue(userId).sortValue(orderId).build();
        return table.getItem(key);
    }

    public List<Order> findByUser(String userId) {
        QueryConditional qc = QueryConditional
            .keyEqualTo(Key.builder().partitionValue(userId).build());
        return table.query(qc).items()
            .stream().collect(Collectors.toList());
    }
}

```

6 When to Use DynamoDB

Strengths, limitations, and decision guide

DynamoDB is the right choice when:

- ✓ **You need predictable, single-digit millisecond latency at any scale** — gaming leaderboards, shopping carts, session stores.
- ✓ **Your access patterns are known and simple** — always query by user, always query by order ID. Avoid ad-hoc analytics.
- ✓ **You are all-in on AWS** — DynamoDB integrates tightly with Lambda, API Gateway, SNS, SQS, and AppSync.
- ✓ **You want zero operational overhead** — no database servers to patch, no replication to configure, no backups to schedule.
- ✓ **Traffic is highly variable** — On-Demand pricing scales to zero at night and to millions of requests during peaks.

Choose a different database when:

- ✗ **You need complex queries or JOINS** — use PostgreSQL or Aurora instead.
- ✗ **Your access patterns are unknown or changing** — DynamoDB requires upfront query design. Use MongoDB for flexibility.
- ✗ **You need full-text search** — DynamoDB does not support it. Use Elasticsearch/OpenSearch alongside DynamoDB.
- ✗ **You are multi-cloud or on-premises** — DynamoDB is AWS-only. Use Cassandra for vendor-neutral deployments.
- ✗ **Cost is unpredictable due to scan-heavy workloads** — Scans are expensive in DynamoDB.

Companies using DynamoDB in production:

Company	Use Case	Scale
Amazon.com	Shopping cart, order management, session store	Billions of requests/day
Lyft	Ride matching, driver/rider state, surge events	Millions of req/sec peaks
Duolingo	User progress, streaks, lesson tracking	500M+ users
Airbnb	Listing availability, booking state machine	Global, multi-region
Samsung	SmartThings device state, IoT telemetry	100B+ device events/day

Key Rules

- Design tables around access patterns first — then choose keys. Never the other way around.
- Use composite sort keys (e.g., 'STATUS#DATE') to enable range queries without a GSI.
- Prefer On-Demand billing for unpredictable workloads — avoids throttling on traffic spikes.

- Use DynamoDB Streams + Lambda for event-driven architectures (e.g., order placed -> send email).
- Use TTL (Time To Live) to auto-expire old items — sessions, temp tokens, rate limit counters.